

Name: A Nandini

Hall ticket No: 2303A51512

Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week9 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 17.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts	Week9 - Wednesday																		

Lab Objectives:

- Learn how to clean raw datasets using AI-assisted Python scripting.
- Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization.
- Automate repetitive data-cleaning tasks with AI-generated code.
- Understand how preprocessing impacts model performance.

Task 1 Employee Dataset Cleaning

Task:

Use AI assistance to clean an employee information dataset.

Instructions:

- Handle missing values in salary, designation, and experience
- Remove duplicate employee IDs
- Standardize text values (job titles in lowercase)
- Encode categorical features (designation, employment_type)

Expected Output:

A clean dataset suitable for HR analytics and prediction tasks.

```

C:\Users\pradeep> cd C:\Users\pradeep> python language > classnode.py > _
1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder
3
4 # Sample employee dataset (replace with your actual data loading)
5 data = {
6     'employee_id': [1, 2, 3, 4, 5, 1, 6],
7     'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', 'Frank'],
8     'salary': [50000, None, 60000, 55000, None, 50000, 65000],
9     'designation': ['Manager', 'Developer', None, 'manager', 'Analyst', 'Manager', 'developer'],
10    'experience': [5, 3, None, 4, 2, 5, 6],
11    'employment_type': ['Full-time', 'Part-time', 'Full-time', 'Contract', 'Full-time', 'Full-time', 'Part-time']
12 }
13
14 df = pd.DataFrame(data)
15
16 # Step 1: Remove duplicate employee IDs (keep the first occurrence)
17 df = df.drop_duplicates(subset='employee_id', keep='first')
18
19 # Step 2: Handle missing values
20 # For salary and experience: fill with median
21 df['salary'] = df['salary'].fillna(df['salary'].median())
22 df['experience'] = df['experience'].fillna(df['experience'].median())
23
24 # For designation: fill with mode (most frequent)
25 df['designation'] = df['designation'].fillna(df['designation'].mode()[0])
26
27 # Step 3: Standardize text values (job titles to lowercase)
28 df['designation'] = df['designation'].str.lower()
29
30 # Step 4: Encode categorical features
31 le_designation = LabelEncoder()
32 df['designation_encoded'] = le_designation.fit_transform(df['designation'])
33
34 le_employment = LabelEncoder()
35 df['employment_type_encoded'] = le_employment.fit_transform(df['employment_type'])
36
37 # Display the clean dataset
38 print(df)
39
40 # Optional: Save to CSV
41 df.to_csv('clean_employee_dataset.csv', index=False)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

Active code page: 65001

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\prade\OneDrive\Pradeep\python language\classnode.py"
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
Active code page: 65001

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\prade\OneDrive\Pradeep\python language\classnode.py"
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
2 3 Charlie 60000.0 analyst 4.0 Full-time 0 1
2 3 Charlie 60000.0 analyst 4.0 Full-time 0 1
3 4 David 55000.0 manager 4.0 Contract 2 0
4 5 Eve 57500.0 analyst 2.0 Full-time 0 1
6 6 Frank 65000.0 developer 6.0 Part-time 1 2

C:\Users\prade>
```

Task 2 – Student Performance Data Preprocessing

Task:

Preprocess student performance data using AI assistance.

Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

Expected Output:

A transformed dataset with time-based academic features and normalized marks.

```
index.html style.css script.js layout.html layoutstyle.css cta.html JS cta.js cta.css Skill_tracker.py

C:\Users\prade>OneDrive\Pradeep\python language> classnode.py ...
1
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import MinMaxScaler
5
6 # Sample Data
7 np.random.seed(42)
8 dates = pd.date_range('2023-01-01', periods=100, freq='D')
9 df = pd.DataFrame({
10     'student_id': range(1, 101),
11     'exam_date': np.random.choice(dates, 100),
12     'total_marks': np.random.normal(75, 15, 100),
13 })
14 df.loc[5, 'total_marks'] = 150 # Outlier
15
16 # 1. Convert to datetime
17 df['exam_date'] = pd.to_datetime(df['exam_date'])
18
19 # 2. Extract time features
20 df['month'] = df['exam_date'].dt.month
21 df['day'] = df['exam_date'].dt.day
22 df['semester'] = df['exam_date'].dt.month.apply(lambda x: 1 if x <= 6 else 2)
23
24 # 3. Outlier Detection - IQR Method
25 Q1, Q3 = df['total_marks'].quantile([0.25, 0.75])
26 IQR = Q3 - Q1
27 lower_bound = Q1 - 1.5 * IQR
28 upper_bound = Q3 + 1.5 * IQR
29 df['outlier'] = (df['total_marks'] < lower_bound) | (df['total_marks'] > upper_bound)
30
31 # 4. Outlier Detection - Z-Score Method
32 z_scores = np.abs((df['total_marks'] - df['total_marks'].mean()) / df['total_marks'].std())
33 df['z_outlier'] = z_scores > 3
34
35 # 5. Handle Outliers (Remove)
36 df = df[~df['outlier']].reset_index(drop=True)
37
38 # 6. Normalize Marks (Min-Max)
39 scaler = MinMaxScaler()
40 df['total_marks_normalized'] = scaler.fit_transform(df[['total_marks']])
41
42 # Display Results
43 print("Preprocessed Data:")
44 print(df[['student_id', 'exam_date', 'month', 'semester', 'total_marks', 'total_marks_normalized']].head(10))
45 print(f"Original shape: {df.shape}, Processed shape: {df.shape}")
46 print(f"Normalized marks range: [{df['total_marks_normalized'].min():.4f}, {df['total_marks_normalized'].max():.4f}]")
47
48 # Save
49 df.to_csv(r'c:\Users\prade\OneDrive\Pradeep\python language\processed_student_data.csv', index=False)
50 print("\n Processed data saved to processed_student_data.csv")
```

```
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\prade\OneDrive\Pradeep\python language\classnode.py"
Preprocessed Data:
student_id exam_date month semester total_marks total_marks_normalized
0 1 2023-02-21 2 1 54.799829 0.161188
1 2 2023-04-03 4 1 61.791131 0.386181
2 3 2023-01-15 1 1 58.041715 0.239148
3 4 2023-03-13 3 1 77.016433 0.578383
4 5 2023-03-02 3 1 83.731042 0.690443
5 7 2023-03-24 3 1 88.424805 0.702269
6 8 2023-03-28 3 1 86.324967 0.744804
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
6 8 2023-03-28 3 1 86.324967 0.744804
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
Original shape: (100,), Processed shape: (97, 9)

Normalized marks range: [0.0000, 1.0000]
✓ Processed data saved to processed_student_data.csv
```

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

```
C:\Users\prade>C:\Users\prade\OneDrive\Pradeep\python language> classnode.py
# Importing student health data
# Preprocess data
df = pd.DataFrame()
# student_id: range(1, 100),
age: np.random.randint(18, 25, 100),
height: np.random.normal(170, 10, 100),
weight: np.random.normal(70, 15, 100),
bmi: np.random.normal(24, 5, 100),
medical_condition: np.random.choice(['Healthy', 'Asthma', 'Diabetes', 'None'], 100),
fitness_status: np.random.choice(['Excellent', 'Good', 'Fair', 'Poor'], 100),
exercise_hours: np.random.uniform(0, 10, 100),
})

# Add missing values
missing_idx = np.random.choice(100, 10, replace=True)
df.loc[missing_idx(1), 'height'] = np.nan
df.loc[missing_idx(3), 'weight'] = np.nan
df.loc[missing_idx(5), 'bmi'] = np.nan

print("Before preprocessing:")
print(f"Missing values: {df.isnull().sum()}")

# 1. Handle Missing Values
df['height'].fillna(df['height'].mean(), inplace=True)
df['weight'].fillna(df['weight'].mean(), inplace=True)
df['bmi'].fillna(df['bmi'].mean(), inplace=True)
df['medical_condition'].fillna('None', inplace=True)
df['fitness_status'].fillna('Fair', inplace=True)

# 2. Encode categorical features
# Medical condition
le_medical = LabelEncoder()
df['medical_encoded'] = le_medical.fit_transform(df['medical_condition'])
# Fitness status
le_fitness = LabelEncoder()
df['fitness_encoded'] = le_fitness.fit_transform(df['fitness_status'])

# 3. Scale numerical features
numerical_cols = ['age', 'height', 'weight', 'bmi', 'exercise_hours']
scaler = StandardScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# 4. Split dataset
X = df[numerical_encoded_cols + ['medical_encoded', 'fitness_encoded']]
y = df['bmi']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("After preprocessing:")
print(f"Missing values: {df.isnull().sum()}")
print(f"Encoded features: {df['medical_encoded'].sum()}")
print(f"Training set: {len(X_train)}, Testing set: {len(X_test)}")

# Save
df.to_csv('c:\Users\prade\OneDrive\Pradeep\python language\processed_health_data.csv', index=False)
X_train.to_csv('c:\Users\prade\OneDrive\Pradeep\python language\health_train.csv', index=False)
X_test.to_csv('c:\Users\prade\OneDrive\Pradeep\python language\health_test.csv', index=False)

print("\n Data saved: processed_health_data.csv, health_train.csv, health_test.csv")
```

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\prade\OneDrive\Pradeep\python language\classnode.py"

Before preprocessing:

Missing values:

student_id	0
age	0
height	3
weight	3
bmi	4
medical_condition	21
fitness_status	25
exercise_hours	0
dtype:	int64

```

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
df['fitness_status'].fillna('Fair', inplace=True)
After preprocessing:
Missing values: 56
Encoded features: 2
Scaled features: 5

Training set: 80, Testing set: 20

   age   height  weight    bmi  exercise_hours  medical_encoded  fitness_encoded
55  1.465198  0.306841  0.023128  0.306740      -0.473295           1             2
88  0.414877 -1.596923 -0.885985 -0.042759      0.568665           3             1
26 -0.110284  0.412532 -0.387245 -0.502624      0.135869           3             4
42  1.465198 -0.817847 -0.364781  2.263329     -1.788820           1             0
69  1.465198  0.004662  1.032667  1.151545     -1.483843           2             2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
After preprocessing:
Missing values: 56
Encoded features: 2
Scaled features: 5

Training set: 80, Testing set: 20

   age   height  weight    bmi  exercise_hours  medical_encoded  fitness_encoded
55  1.465198  0.306841  0.023128  0.306740      -0.473295           1             2
88  0.414877 -1.596923 -0.885985 -0.042759      0.568665           3             1
26 -0.110284  0.412532 -0.387245 -0.502624      0.135869           3             4
42  1.465198 -0.817847 -0.364781  2.263329     -1.788820           1             0
69  1.465198  0.004662  1.032667  1.151545     -1.483843           2             2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
42  1.465198 -0.817847 -0.364781  2.263329     -1.788820           1             0
69  1.465198  0.004662  1.032667  1.151545     -1.483843           2             2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv

```

Task 4: Real-Time Application: Ride-Sharing Data Cleaning Scenario:

A ride-sharing company has trip data with errors.

Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

```

C:\Users> python3 .\task4.py
6 # Sample Ride-Sharing Data
7 np.random.seed(42)
8
9 data = {
10     'ride_id': range(1, 100),
11     'pickup_location': np.random.choice(['downtown', 'airport', 'downtown', 'airport', 'home', 100),
12     'drop_location': np.random.choice(['downtown', 'airport', 'downtown', 'airport', 'home', 100),
13     'fare_amount': np.random.normal(15, 10, 100),
14     'driver_rating': np.random.uniform(1, 5, 100),
15 }
16
17 df = pd.DataFrame(data)
18 # Add duplicates and missing values
19 df.loc[random_rows] = np.random.choice(100, 5, replace=False), 'fare_amount' = np.nan
20 df = df.reset_index(drop=True)
21
22 print("RIDE-SHARING DATA CLEANING PIPELINE")
23 print("="*40)
24 print(f"Original shape: {df.shape}")
25 print(f"Missing values: {df.isnull().sum()}")
26
27 # 1. Standardize Locations
28 df['pickup_location'] = df['pickup_location'].str.strip().str.lower().str.title()
29 df['drop_location'] = df['drop_location'].str.strip().str.lower().str.title()
30 df['pickup_location'] = df['pickup_location'].fillna('unknown', inplace=True)
31 df['drop_location'] = df['drop_location'].fillna('unknown', inplace=True)
32 print(f"Standardized pickup and drop locations")
33
34 # 2. Fill Missing Fares with Median
35 median_fare = df['fare_amount'].median()
36 df['fare_amount'] = df['fare_amount'].fillna(median_fare, inplace=True)
37 print(f"Filled missing fares with median: {median_fare:.2f}")
38
39 # 3. Remove Duplicates
40 initial_rows = len(df)
41 df = df.drop_duplicates(subset=['ride_id', 'pickup_location', 'drop_location'], keep='first')
42 df = df.reset_index(drop=True)
43 duplicates_removed = initial_rows - len(df)
44 print(f"Removed duplicate entries: {duplicates_removed} rows")
45
46 # 4. Normalize Driver Ratings (0-1 scale)
47 scaler = MinMaxScaler()
48 df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])
49 print(f"Normalized driver ratings (0-1 scale)")
50
51 # 5. Generate Summary
52 print("\n" + "="*40)
53 print("DATA QUALITY IMPROVEMENT SUMMARY")
54 print("="*40)
55 print(f"Final dataset shape: {df.shape}")
56 print(f"Duplicate entries removed: {duplicates_removed}")
57 print(f"Missing fare amount filled: {df['fare_amount'].isnull().sum() == 0}")
58 print(f"Location standardization: Complete")
59 print(f"Driver ratings normalized: Complete")
60 print(f"Fare amount statistics:")
61 print(f"Min: {df['fare_amount'].min():.2f}")
62 print(f"Max: {df['fare_amount'].max():.2f}")
63 print(f"Mean: {df['fare_amount'].mean():.2f}")
64 print(f"Median: {df['fare_amount'].median():.2f}")

```

```
index.html style.css scripts layout.html layoutstyle.css cta.html cta.js cta.css Skill_tracker.py classnode.py X
C:\Users\prade > OneDrive > Pradeep > python language > classnode.py > ...

64
65 # Save cleaned data
66 df.to_csv(r'c:\Users\prade\OneDrive\Pradeep\python language\cleaned_rides.csv', index=False)
67 print(f'Clean dataset saved to cleaned_rides.csv')
68 print(f'Cleaned Data Sample:\n(df.head())')
69

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\prade\OneDrive\Pradeep\python language\classnode.py"
RIDE-SHARING DATA CLEANING PIPELINE
=====

Original shape: (185, 5)
Missing values:
ride_id      0
pickup_location 22
drop_location 17
fare_amount   5
driver_rating 0
dtype: int64

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
df['fare_amount'].fillna(method='ffill', inplace=True)
✓ Filled missing fares with median: $25.44
✓ Removed duplicate entries: 5 rows
✓ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: False
Location standardization: Complete
Driver ratings normalized: Complete

=====
Fare Statistics:
Min: $11.82
Max: $55.41
Mean: $25.28
Median: $25.28
df['fare_amount'].fillna(method='ffill', inplace=True)
✓ Filled missing fares with median: $25.44
✓ Removed duplicate entries: 5 rows
✓ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: False
Location standardization: Complete
Driver ratings normalized: Complete

=====
Fare Statistics:
Min: $11.82
Max: $55.41
Mean: $25.28
Median: $25.28

=====
✓ Clean dataset saved to cleaned_rides.csv

=====
Fare Statistics:
Min: $11.82
Max: $55.41
Mean: $25.28
Median: $25.28

=====
✓ Clean dataset saved to cleaned_rides.csv

=====
Fare Statistics:
Min: $11.82
Max: $55.41
Mean: $25.28
Median: $25.28

=====
✓ Clean dataset saved to cleaned_rides.csv

=====
Cleaned Data Sample:

Cleaned Data Sample:
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
0 1 Airport Midtown 24.545973 2.312611 0.323688
0 1 Airport Midtown 24.545973 2.312611 0.323688
1 2 NaN Midtown 27.160367 1.628166 0.164375
2 3 Downtown Midtown 36.124237 4.927264 1.000000
1 2 NaN Midtown 27.160367 1.628166 0.164375
2 3 Downtown Midtown 36.124237 4.927364 1.000000
3 4 NaN Station 38.434887 4.355734 0.852145
4 5 NaN Midtown 25.298995 4.441618 0.874359
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.